

Motor de Tráfico Aéreo (Nodos AVL)

Materia: Estructuras de Datos / Programación II

Tiempo Estimado: 4 Horas

1. Introducción

El Aeropuerto Internacional de Santa Ana está modernizando su sistema de radar. Actualmente, los vuelos se procesan en una cola de prioridad ineficiente. Se requiere un **Motor de Gestión de Espacio Aéreo** que organice los vuelos según su **altitud (pies)** para detectar conflictos de tráfico rápidamente.

Se te entrega un **Código Base** que implementa la estructura de un Árbol AVL. Tu misión es auditar la seguridad de memoria de Rust, implementar la lógica de "aterrizaje" (eliminación) y expandir el sistema con análisis de proximidad.

2. Requisitos del Ejercicio

FASE 1: Análisis de Seguridad y Propiedad (45 min)

Antes de modificar el radar, debes entender cómo Rust protege los datos:

1. **Documentación de Memoria:** Explica en el código base qué ocurre con la propiedad (ownership) cuando se usa `Option::take()` dentro de las rotaciones.
2. **Prueba de Escritorio:** Dibuja el árbol AVL resultante tras insertar vuelos con las siguientes altitudes: `[5000, 3000, 2000, 4000, 3500, 6000]`. Identifica el tipo de rotación (Simple o Doble) realizada en cada paso crítico.
3. **Concepto de Box:** ¿Por qué usamos `Box<Nodo>` en lugar de `Nodo` directamente dentro de la estructura? Explica la relación con el tamaño en memoria en tiempo de compilación.

FASE 2: Localización de Vuelos (60 min)

Implementa una función para encontrar un vuelo específico por su altitud.

- **Firma:** `fn buscar_vuelo(nodo: &Option<Box<Nodo>>, altitud: u32) -> Option<&Vuelo>`
- **Restricción:** El motor debe ser de solo lectura (usar referencias), garantizando que el radar no modifique accidentalmente los datos del vuelo.

FASE 3: Descenso y Aterrizaje (Eliminación) (90 min)

Cuando un avión aterriza, debe ser removido del árbol de altitud. Esta es la operación más crítica.

- **Firma:** `fn eliminar_vuelo(nodo_opt: Option<Box<Nodo>>, altitud: u32) -> Option<Box<Nodo>>`

- **Desafío:** Si el nodo tiene dos hijos, utiliza el **predecesor in-order** (el valor más alto del subárbol izquierdo) para sustituirlo.
- **Validación:** Tras la eliminación, el árbol debe recalcular alturas y ejecutar las rotaciones necesarias para permanecer balanceado.

FASE 4: Alerta de Colisión (45 min)

Elige **UNA** de las siguientes funciones avanzadas:

- **Opción A (Alerta de Proximidad):** `fn vuelos_en_rango(nodo: &Option<Box<Nodo>>, min: u32, max: u32) -> usize`. Cuenta cuántos aviones están volando peligrosamente cerca (en un rango de altitud dado).
- **Opción B (Vuelo de Emergencia):** Una función que encuentre y retorne el vuelo con la **menor altitud** (el más cercano a tierra) sin recorrer todo el árbol.

3. Rúbrica de Evaluación (10 Puntos)

Criterio	Excelente (2 pts)	Regular (1 pt)	Insuficiente (0 pts)
Gestión de Memoria	Uso impecable de <code>take()</code> , <code>as_ref()</code> y manejo de <i>Ownership</i> .	El código compila pero usa <code>.clone()</code> innecesarios.	Errores de compilación por el <i>Borrow Checker</i> .
Lógica de Eliminación	Maneja los 3 casos y el re-balanceo es correcto.	Elimina el nodo pero deja el árbol desbalanceado.	La eliminación corrompe la estructura.
Eficiencia de Búsqueda	Implementación $O(\log n)$ usando referencias.	Realiza copias de los objetos o busca en $O(n)$.	No implementado o no funcional.
Prueba de Escritorio	Dibujo preciso con rotaciones identificadas.	Árbol final correcto pero sin explicar rotaciones.	Resultado incorrecto.

Calidad del Código	Código limpio, indentado y con comentarios útiles.	Código difícil de leer o sin comentarios.	Código desordenado.
---------------------------	--	---	---------------------

4. Código Base (Radar Inicial)

Rust

None

```
#[derive(Debug, Clone)]
struct Vuelo {
    id: String,
    altitud: u32, // Este será nuestra clave (key)
}

struct Nodo {
    vuelo: Vuelo,
    izquierdo: Option<Box<Nodo>>,
    derecho: Option<Box<Nodo>>,
    altura: i32,
}

impl Nodo {
    fn nuevo(vuelo: Vuelo) -> Self {
        Nodo {
            vuelo,
            izquierdo: None,
            derecho: None,
            altura: 1,
        }
    }
}

// --- UTILIDADES DE BALANCEO (NO MODIFICAR) ---

fn obtener_altura(nodo: &Option<Box<Nodo>>) -> i32 {
    nodo.as_ref().map_or(0, |n| n.altura)
}
```

```

fn actualizar_altura(nodo: &mut Nodo) {
    nodo.altura = 1 + std::cmp::max(
        obtener_altura(&nodo.izquierdo),
        obtener_altura(&nodo.derecho),
    );
}

fn obtener_balance(nodo: &Nodo) -> i32 {
    obtener_altura(&nodo.izquierdo) - obtener_altura(&nodo.derecho)
}

fn rotar_derecha(mut y: Box<Nodo>) -> Box<Nodo> {
    let mut x = y.izquierdo.take().expect("Error de radar");
    y.izquierdo = x.derecho.take();
    actualizar_altura(&mut y);
    x.derecho = Some(y);
    actualizar_altura(&mut x);
    x
}

fn rotar_izquierda(mut x: Box<Nodo>) -> Box<Nodo> {
    let mut y = x.derecho.take().expect("Error de radar");
    x.derecho = y.izquierdo.take();
    actualizar_altura(&mut x);
    y.izquierdo = Some(x);
    actualizar_altura(&mut y);
    y
}

// --- FUNCIÓN DE INSERCIÓN ---

fn insertar(nodo_opt: Option<Box<Nodo>>, vuelo: Vuelo) -> Box<Nodo>
{
    let mut nodo = match nodo_opt {
        None => return Box::new(Nodo::nuevo(vuelo)),
        Some(n) => n,
    }
}

```

```

    };

    if vuelo.altitud < nodo.vuelo.altitud {
        nodo.izquierdo = Some(insertar(nodo.izquierdo.take(),
vuelo));
    } else if vuelo.altitud > nodo.vuelo.altitud {
        nodo.derecho = Some(insertar(nodo.derecho.take(), vuelo));
    } else {
        return nodo;
    }

    actualizar_altura(&mut nodo);
    let balance = obtener_balance(&nodo);

    // Caso Izquierda-Izquierda
    if balance > 1 && vuelo.altitud <
nodo.izquierdo.as_ref().unwrap().vuelo.altitud {
        return rotar_derecha(nodo);
    }
    // Caso Derecha-Derecha
    if balance < -1 && vuelo.altitud >
nodo.derecho.as_ref().unwrap().vuelo.altitud {
        return rotar_izquierda(nodo);
    }
    // Caso Izquierda-Derecha
    if balance > 1 && vuelo.altitud >
nodo.izquierdo.as_ref().unwrap().vuelo.altitud {
        let hijo_izq = nodo.izquierdo.take().unwrap();
        nodo.izquierdo = Some(rotar_izquierda(hijo_izq));
        return rotar_derecha(nodo);
    }
    // Caso Derecha-Izquierda
    if balance < -1 && vuelo.altitud <
nodo.derecho.as_ref().unwrap().vuelo.altitud {
        let hijo_der = nodo.derecho.take().unwrap();
        nodo.derecho = Some(rotar_derecha(hijo_der));
    }

```

```

        return rotar_izquierda(nodo);
    }

    nodo
}

fn main() {
    let mut radar: Option<Box<Nodo>> = None;

    // Simulación de entrada de vuelos
    let datos = vec![
        ("AV123", 5000), ("UA456", 3000), ("IB101", 2000),
        ("AF999", 4000), ("TA222", 3500), ("AM777", 6000),
    ];

    for (id, alt) in datos {
        let v = Vuelo { id: id.to_string(), altitud: alt };
        radar = Some(insertar(radar.take(), v));
    }

    println!("--- Radar de Control Aéreo (AVL) ---");
    // Aquí el estudiante debe invocar sus funciones de búsqueda y
    eliminación
}

```